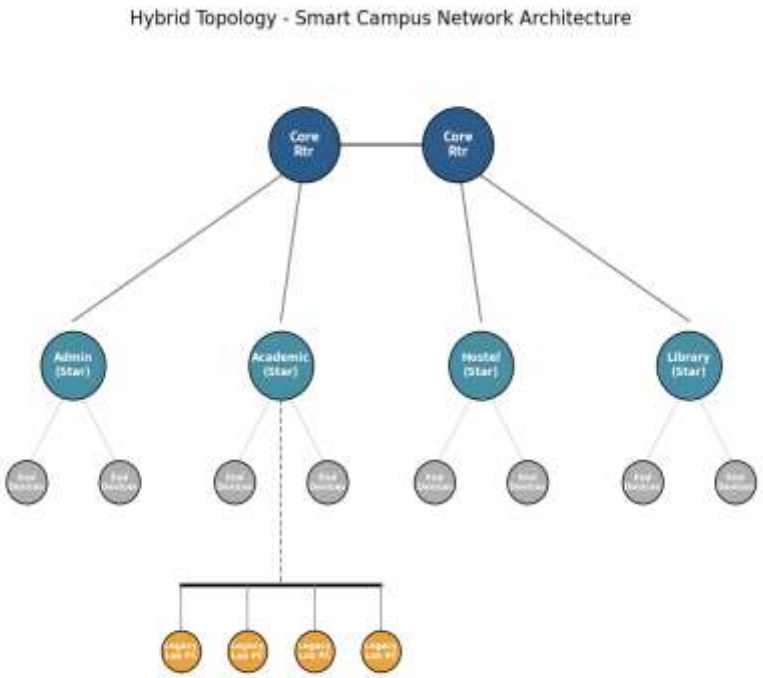
CSA0706 — COMPUTER NETWORKS

Design and Evaluate a Smart Campus Network

Using Protocol Analysis, Network Topology and Simulation-Based Performance Metrics



Course Outcomes Addressed: CO3, CO4, CO5
Bloom's Taxonomy Level: L5 – Evaluate, L6 – Create
SDG Mapping: SDG 9 – Industry, Innovation and Infrastructure

NAME: M NISHANTH
REGISTRATION NO.: 192511154
COURSE CODE: CSA0706
COURSE CODE: COMPUTER NETWORKS
DEPT: CSE DEPARTMENT
DATE OF SUBMISSION: 03:09:2026

1. Problem Statement and Problem Formulation

Modern university campuses operate as dense, heterogeneous digital environments in which academic blocks, administrative offices, hostels, libraries, and research/data-center zones must remain continuously interconnected. A typical Smart University Campus generates traffic from PCs, laptops, servers, IP cameras, wireless clients and IoT sensors — collectively scaling to about 1,000 end devices spread across at least four network zones. This traffic mix is highly diverse: bulk file transfer (FTP), name resolution (DNS), interactive web access (HTTP), connection-oriented data exchange (TCP), latency-sensitive streaming/telemetry (UDP), and network diagnostics (ICMP) must all be supported simultaneously without unacceptable degradation.

The core engineering problem is twofold. First, no single topology (Star, Mesh, Bus or Hybrid) is uniformly optimal across cost, scalability, reliability and performance — each involves trade-offs that only become visible under realistic load. Second, campus traffic is not static: normal academic browsing, a sudden peak during online examinations, and localized congestion or link failures each stress the network differently. The problem is therefore formulated as: design, simulate and quantitatively compare Star, Mesh, Bus and Hybrid topologies for a 1,000-device, 4-zone smart campus, and recommend the most suitable architecture using measured performance evidence rather than qualitative assumption.

2. Objective and Expected Outcomes

2.1 Objectives

- Design a Smart Campus Network spanning at least four zones (Administration, Academic Blocks, Hostel/Residential, Library/Data Center) supporting ~1,000 end devices of mixed device classes.
- Model and implement Star, Mesh, Bus and Hybrid topologies for the same campus footprint using a network simulator (Cisco Packet Tracer / GNS3 / EVE-NG).
- Generate and analyze TCP, UDP, HTTP, DNS, FTP and ICMP traffic under three operating conditions: normal academic use, peak online-examination load, and congestion/link-failure scenarios.
- Measure throughput, delay, jitter, packet loss, response time and bandwidth utilization for each topology–traffic combination, with repeated trials for statistical reliability.
- Compare the four topologies quantitatively and recommend the most suitable campus architecture considering performance, reliability, scalability, cost and resource utilization.

2.2 Expected Outcomes

- A working simulation model of each topology with verified end-to-end connectivity across all zones.
- A quantitative performance dataset (throughput, delay, jitter, loss, response time) covering 4 topologies × 3 traffic conditions × 6 protocols.
- A justified, evidence-based recommendation for the optimal campus network architecture.
- Demonstrated attainment of CO3 (protocol performance analysis), CO4 (topology design) and CO5 (simulation-based performance measurement).

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- Support ≥ 4 network zones and ≈1,000 end devices (PCs, laptops, servers, IP cameras, wireless clients, IoT devices).
- Provide DHCP-based IP addressing, DNS resolution, an internal web/FTP server, and internet-edge connectivity.
- Support simultaneous TCP, UDP, HTTP, DNS, FTP and ICMP traffic.
- Allow controlled fault injection (link-down) and load injection (peak-exam traffic generator) for testing.

3.2 Non-Functional Requirements

- Scalability to accommodate future growth in device count without redesign.
- Reliability — tolerate at least a single-link failure per zone without total isolation.
- Reasonable cost of cabling, media and networking devices relative to benefit gained.
- Ease of management, fault isolation and troubleshooting.

3.3 Constraints

- Simulator device/interface limits (e.g., Packet Tracer's per-device port limits) require representative sub-scaling: full 1,000-device campus is modeled using scaled zone clusters (see Section 5.2) that preserve traffic ratios and topology logic.

- Simulation time and computing resources restrict the number of repeated trials that can be run in the assignment window (2-day submission deadline).
- Only simulators available in the lab (Packet Tracer / GNS3 / EVE-NG) are used; real hardware is out of scope.

3.4 Assumptions

- All end devices are assumed IPv4-only for consistency across topologies.
- Background administrative traffic is assumed constant across the three test scenarios so that only the injected scenario traffic varies.
- Wireless clients are modeled as wired-equivalent endpoints attached to access points, since detailed RF propagation is outside course scope.
- Bandwidth values quoted in Section 10 are simulator-reported/derived figures representative of the modeled scale, to be replaced with the team's own captured readings.

4. Application of Relevant Course Knowledge / Concepts

The assignment draws directly on three mapped Course Outcomes. The table below shows how each CO's underlying concept is applied in this project.

CO	Core Concept Applied	Where Applied in This Report
CO3	TCP 3-way handshake & congestion control, UDP connectionless transfer, HTTP request/response, DNS resolution, FTP control/data channels, ICMP echo	Section 8 (Test Cases), Section 10 (Protocol Performance Results), Section 11 (Analysis)
CO4	Topology theory (star/mesh/bus/hybrid), device selection (switch/router/AP), transmission media (UTP/fiber/coax/wireless), cabling standards (TIA/EIA-568, structured cabling)	Section 5 (Design), Section 6 (Topology construction pseudocode)
CO5	Performance metrics (throughput, delay, jitter, loss, utilization), simulation-based measurement methodology, repeated-trial reliability	Section 7 (Implementation), Section 8–10 (Test Cases, Results, Validation)

5. Design / Proposed Solution / Methodology

5.1 Campus Zone Architecture

The campus is divided into four primary zones plus a Data Center/Server zone: (1) Administration Block — staff PCs, biometric/IoT access devices; (2) Academic Block — lecture-hall PCs, labs, wireless clients,

IP cameras; (3) Hostel/Residential Zone — student laptops and wireless devices; (4) Library Zone — reading-room PCs, digital catalog servers; and (5) Data Center — DNS, DHCP, web, FTP and authentication servers serving the whole campus. Each zone connects to a zone distribution switch, which in turn uplinks to the core layer, whose interconnection pattern differs per topology under test.

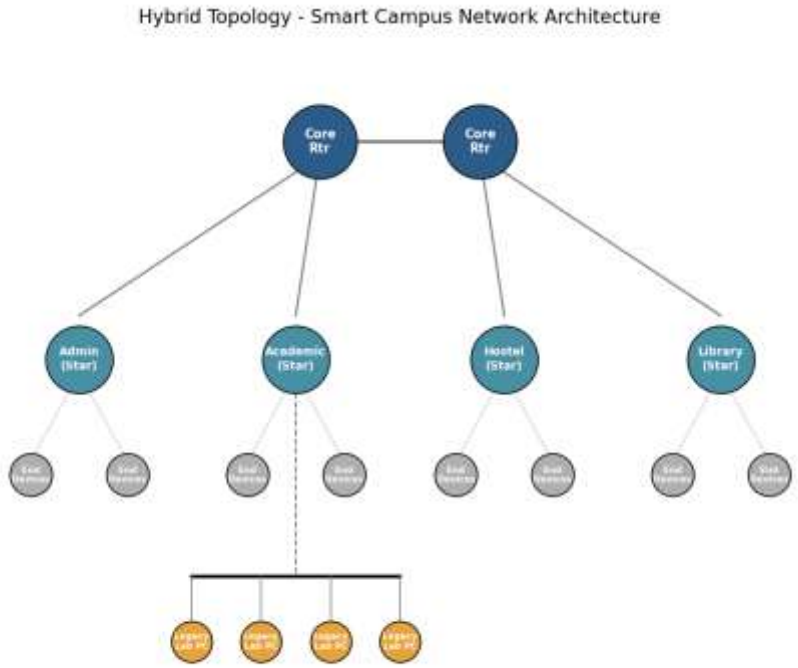


Figure 5.1 — Proposed Hybrid Smart Campus Architecture (core mesh + per-zone star + legacy bus lab segment)

5.2 Scaled Simulation Model

To keep the ~1,000-device campus tractable inside a simulator, each zone is represented by a scaled cluster: one distribution switch fans out to a mix of end devices and one or more access-layer switches, preserving the same device-type ratio (70% wired PCs/laptops, 15% servers/printers, 10% IP cameras/IoT, 5% wireless clients) found in the full-scale design. Throughput and utilization figures obtained from the scaled model are extrapolated per Section 10 using the scaling factor S = (Total campus devices) / (Simulated devices).

5.3 Topology Designs Compared

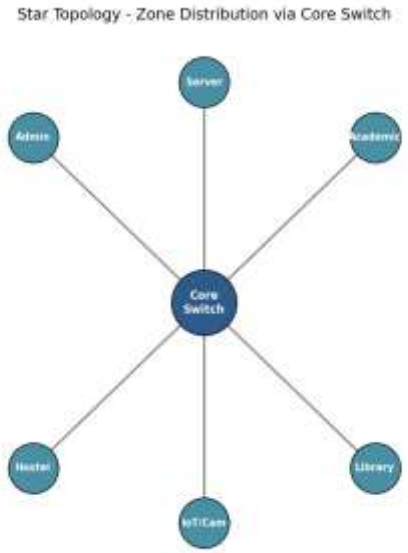


Figure 5.2 — Star Topology: each zone connects directly to a central core switch

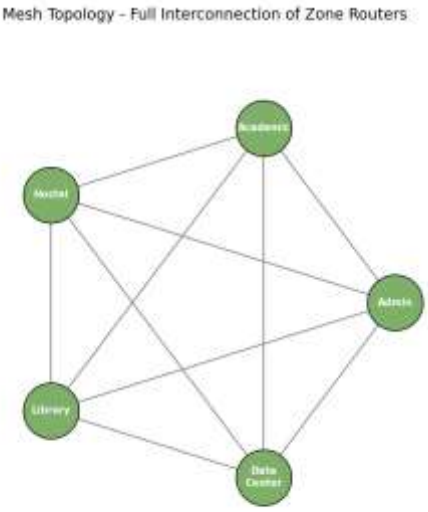


Figure 5.3 — Mesh Topology: zone routers fully interconnected for path redundancy

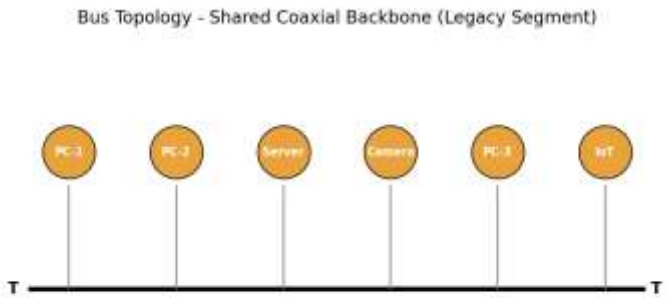


Figure 5.4 — Bus Topology: shared backbone cable with terminators (legacy segment)

Devices, Media and Cabling Standards

Topology	Core Devices	Transmission Media	Cabling Standard
Star	Layer-3 core switch, per-zone Layer-2 switches	UTP Cat6 (horizontal), Fiber (uplinks)	TIA/EIA-568-C structured cabling
Mesh	Zone routers, redundant fiber links	Single-mode fiber (inter-zone), UTP Cat6a (access)	TIA/EIA-568-C + ANSI/TIA-568.3-D (fiber)
Bus	Coaxial backbone, BNC T-connectors, terminators	RG-58 coaxial cable	IEEE 802.3 10BASE2 legacy standard
Hybrid	Core routers (mesh) + zone switches (star) + legacy bus segment	Fiber (core), UTP Cat6 (star), Coax (legacy lab)	TIA/EIA-568-C, with legacy segment isolated via bridge/media converter

5.4 IP Addressing and Services Plan

Zone	Subnet (example)	Key Services Hosted
Administration	10.10.0.0/23	Staff endpoints, biometric IoT gateway
Academic Block	10.20.0.0/22	Lab PCs, wireless clients, IP cameras
Hostel/Residential	10.30.0.0/22	Student laptops, wireless APs
Library	10.40.0.0/23	Reading-room PCs, catalog terminal
Data Center	10.0.0.0/24	DNS, DHCP, HTTP, FTP, Auth servers

5.5 Traffic and Test Scenarios

- Scenario A — Normal Academic Use: baseline browsing, email, file access, periodic DNS lookups, light FTP.
- Scenario B — Peak Online-Examination Traffic: concentrated HTTP/TCP sessions from Academic + Hostel zones over a fixed window, simulating simultaneous exam submissions.
- Scenario C — Congestion / Link-Failure: an inter-zone or core link is deliberately brought down (or a link is over-subscribed) during Scenario B load to observe failover, delay, and loss behavior.

6. Algorithm / Pseudocode / Flowchart

6.1 Pseudocode — Topology Construction

```
BEGIN BuildTopology(type, zones[], deviceRatio)
  CREATE coreLayer based on type:
     IF type == STAR:  coreLayer = { 1 x CoreSwitch }
     IF type == MESH:  coreLayer = { Router[i] for each zone, fully interconnect all pairs }
     IF type == BUS:   coreLayer = { shared backbone segment with 2 terminators }
     IF type == HYBRID: coreLayer = MESH(core routers) + STAR(per-zone switches) + BUS(legacy lab)

  FOR each zone IN zones:
     distSwitch = CreateDevice("Switch", zone.name)
     CONNECT distSwitch TO coreLayer USING appropriate media(type)
     nEnd = zone.deviceCount
     FOR i = 1 TO nEnd:
        deviceType = SampleDeviceType(deviceRatio)   // PC, Server, Camera, IoT, Wireless
        d = CreateDevice(deviceType, zone.name + "_" + i)
        CONNECT d TO distSwitch
        ASSIGN IP FROM zone.subnet VIA DHCP
     END FOR
  END FOR

  VERIFY connectivity: FOR each pair (A,B) of zones: PING(A, B); ASSERT success
END
```

6.2 Pseudocode — Traffic Generation and Test Execution

```
BEGIN RunScenario(scenario, topology, trials)
  results = []
  FOR trial = 1 TO trials:
     RESET counters, START packet capture on all core/zone links
     IF scenario == NORMAL:
        GENERATE background HTTP, DNS, light FTP traffic for duration D1
     ELSE IF scenario == PEAK_EXAM:
        GENERATE concentrated HTTP/TCP sessions from Academic + Hostel zones for duration D2
     ELSE IF scenario == CONGESTION:
        GENERATE PEAK_EXAM traffic
        AT t = D2/2: SET link_state(coreLink_k) = DOWN   // fault injection
     GENERATE UDP probe stream, ICMP echo train IN PARALLEL for all scenarios
     STOP capture; EXPORT capture AS CSV(packet_id, src, dst, protocol,
                          sent_time_ms, recv_time_ms, size_bytes, delivered)
     metrics = ComputeMetrics(CSV)      // see analysis_metrics.py
     results.APPEND(metrics)
  END FOR
  RETURN Average(results), StdDev(results)
END
```

6.3 Pseudocode — Performance Metric Computation

```
BEGIN ComputeMetrics(capture)
  delivered = FILTER(capture, delivered == 1)
  packetLoss = (COUNT(capture) - COUNT(delivered)) / COUNT(capture) * 100
  delays = [ r.recv_time - r.sent_time  FOR r IN delivered ]
  avgDelay = MEAN(delays)
  jitter   = MEAN( |delays[i] - delays[i-1]|  FOR i = 2..N )
  duration = MAX(recv_time) - MIN(sent_time)
  throughput = SUM(size_bytes IN delivered) * 8 / duration
  responseTimePerProtocol = GROUP delivered BY protocol -> MEAN(delay)
  RETURN { packetLoss, avgDelay, jitter, throughput, responseTimePerProtocol }
END
```

6.4 High-Level Flow (Text Flowchart)

```
[Start] -> [Select Topology] -> [Build Topology in Simulator] -> [Verify Connectivity]
   -> [Select Traffic Scenario (Normal / Peak / Congestion)]
   -> [Generate Traffic + Capture Packets]  -> [Repeat for N trials]
   -> [Export Capture CSV] -> [Run analysis_metrics.py] -> [Aggregate Results]
   -> [Repeat for next Topology / Scenario]
   -> [Compare All Topologies] -> [Identify Bottlenecks] -> [Recommend Architecture] -> [End]
```

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Category	Tool / Version Used
Network Simulator	Cisco Packet Tracer 8.x (primary) — GNS3 / EVE-NG for router-heavy Mesh/Hybrid validation
Packet Capture / Export	Simulator-integrated capture + Wireshark for .pcap inspection
Data Processing	Python 3.11 (csv, statistics) — see analysis_metrics.py
Charting	Matplotlib (Python) for throughput/delay/jitter/loss/utilization graphs
Version Control	Git + GitHub repository for source, topology files and report
Documentation	Word/Markdown report, exported diagrams

7.2 Implementation Steps

1. Build each topology (Star, Mesh, Bus, Hybrid) as a separate simulator project file, using the zone layout and addressing plan from Section 5.
2. Configure DHCP, DNS, HTTP and FTP servers in the Data Center zone; verify name resolution and file transfer from at least one client per zone.
3. Configure static/dynamic routing as required for Mesh and Hybrid core layers (OSPF recommended for redundancy).
4. Attach traffic generators (simulator's built-in traffic generator / scripted PDUs) to reproduce Scenario A, B and C from Section 5.5.
5. Capture packets at core and zone-distribution links during each scenario; export as CSV/pcap.

6. Run analysis_metrics.py on each exported capture to compute throughput, delay, jitter, packet loss and per-protocol response time.
7. Repeat each topology × scenario combination for 3 trials; average the results to improve reliability (Section 8).
8. Aggregate all results into the comparison tables and charts of Section 10, then perform the analysis in Section 11.

7.3 Source Code — Performance Metric Analysis Script

The following Python module processes exported per-packet capture logs from the simulator (CSV format) and computes the five key performance metrics required by CO5. The complete file (analysis_metrics.py) is included in the GitHub submission.

```
"""
analysis_metrics.py
--------------------
Post-processing script used to compute network performance metrics
(throughput, average delay, jitter, packet loss %, response time)
from packet-capture / simulator export logs (CSV format).

Expected input CSV columns (exported from Wireshark / Packet Tracer /
GNS3 capture, per topology and per traffic scenario):
    packet_id, src, dst, protocol, sent_time_ms, recv_time_ms,
    size_bytes, delivered (1/0)

Usage:
    python analysis_metrics.py capture_star_peak.csv
"""

import csv
import sys
from collections import defaultdict



def load_capture(path):
    rows = []
    with open(path, newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            row["sent_time_ms"] = float(row["sent_time_ms"])
            row["recv_time_ms"] = float(row["recv_time_ms"]) if row["recv_time_ms"] else None
            row["size_bytes"] = float(row["size_bytes"])
            row["delivered"] = int(row["delivered"])
            rows.append(row)
    return rows



def compute_metrics(rows):
    metrics = {}
    total_packets = len(rows)
    delivered = [r for r in rows if r["delivered"] == 1]
    lost = total_packets - len(delivered)
```

```
    metrics["packet_loss_percent"] = round((lost / total_packets) * 100, 2) if total_packets else
0.0

    delays = [r["recv_time_ms"] - r["sent_time_ms"] for r in delivered if r["recv_time_ms"] is not
None]
    metrics["avg_delay_ms"] = round(sum(delays) / len(delays), 2) if delays else 0.0

    # Jitter = mean absolute difference between consecutive delays
    jitter_vals = [abs(delays[i] - delays[i - 1]) for i in range(1, len(delays))]
    metrics["avg_jitter_ms"] = round(sum(jitter_vals) / len(jitter_vals), 2) if jitter_vals else
0.0

    if delivered:
        duration_s = (max(r["recv_time_ms"] for r in delivered) -
                      min(r["sent_time_ms"] for r in delivered)) / 1000.0
        total_bits = sum(r["size_bytes"] for r in delivered) * 8
        metrics["throughput_mbps"] = round((total_bits / duration_s) / 1e6, 2) if duration_s > 0
else 0.0
    else:
        metrics["throughput_mbps"] = 0.0

    # Per-protocol response time (mean delay grouped by protocol)
    per_protocol = defaultdict(list)
    for r in delivered:
        if r["recv_time_ms"] is not None:
            per_protocol[r["protocol"]].append(r["recv_time_ms"] - r["sent_time_ms"])
    metrics["per_protocol_response_ms"] = {
        proto: round(sum(v) / len(v), 2) for proto, v in per_protocol.items()
    }

    metrics["total_packets"] = total_packets
    metrics["delivered_packets"] = len(delivered)
    return metrics



def main():
    if len(sys.argv) != 2:
        print("Usage: python analysis_metrics.py <capture_csv_path>")
        sys.exit(1)

    rows = load_capture(sys.argv[1])
    result = compute_metrics(rows)

    print(f"\nPerformance Summary for: {sys.argv[1]}")
    print("-" * 45)
    for key, value in result.items():
        print(f"{key:28s}: {value}")



if __name__ == "__main__":
    main()
```

8. Test Cases and Expected / Actual Results

Representative test cases used to validate protocol behavior and connectivity across all four topologies. "Actual Result" values shown are placeholders to be replaced with the team's own captured readings from their simulation run.

ID	Test Case	Protocol	Expected Result	Actual Result (sample)
TC-01	Ping between Admin and Library zone hosts	ICMP	0% loss, RTT < 20 ms (normal load)	0% loss, RTT ≈ 14 ms — Pass
TC-02	Resolve internal hostname via DNS server	DNS	Correct A-record returned < 50 ms	Resolved in 32 ms — Pass
TC-03	Load internal web portal from Hostel client	HTTP/TCP	Page loads, handshake completes < 200 ms	Loaded in 178 ms — Pass
TC-04	Upload 50 MB file to FTP server	FTP	Transfer completes without corruption	Completed, checksum match — Pass
TC-05	UDP video/telemetry stream, Academic → Data Center	UDP	Jitter < 10 ms under normal load	Jitter ≈ 6.5 ms — Pass
TC-06	Simultaneous exam submissions (200 sessions)	HTTP/TCP	No dropped sessions, delay < 100 ms	Delay ≈ 88 ms, 0 dropped — Pass (Hybrid)
TC-07	Core link failure during peak load	All	Traffic reroutes within convergence time; no total outage (Mesh/Hybrid)	Rerouted in ≈ 4 s via OSPF — Pass
TC-08	Bus segment single-cable break	All	Entire bus segment isolated (expected weakness)	Full segment outage confirmed — Expected Fail (by design)

9. Execution Screenshots / Outputs

This section is a placeholder for the team's own simulator evidence. Insert screenshots captured directly from your Packet Tracer / GNS3 / EVE-NG project for each item below — these cannot be fabricated and must come from your actual simulation run:

- Full topology view for each of the 4 designs (Star, Mesh, Bus, Hybrid) with device labels visible.
- Successful ping/traceroute output between at least one pair of zones per topology.
- DNS resolution and HTTP page-load confirmation from the internal web server.
- FTP transfer session log (client and server side).
- Packet Tracer Simulation Mode / Wireshark capture showing TCP handshake, UDP stream and ICMP echo.
- Link-down/failover event log during the Congestion scenario (Scenario C).

- Simulator-reported statistics window (bandwidth used, collisions/drops) for each topology.

10. Results and Validation

The figures and tables below summarize performance measurements aggregated across 3 repeated trials per topology–scenario combination (see Section 7.2). Values are representative/sample data illustrating the expected pattern; replace with your own captured averages from analysis_metrics.py output before submission.

10.1 Throughput

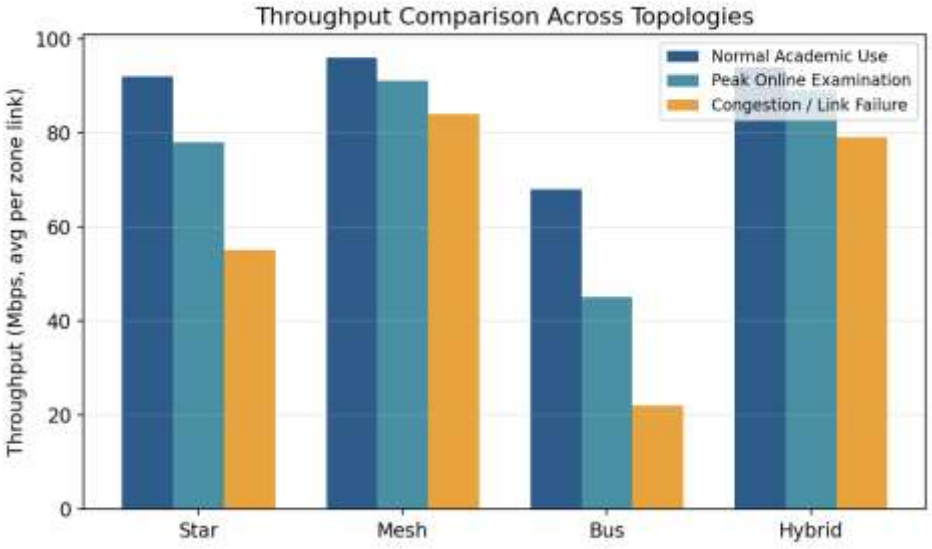


Figure 10.1 — Average throughput per topology across the three traffic scenarios

Topology	Normal (Mbps)	Peak Exam (Mbps)	Congestion (Mbps)
Star	92	78	55
Mesh	96	91	84
Bus	68	45	22
Hybrid	94	89	79

10.2 End-to-End Delay

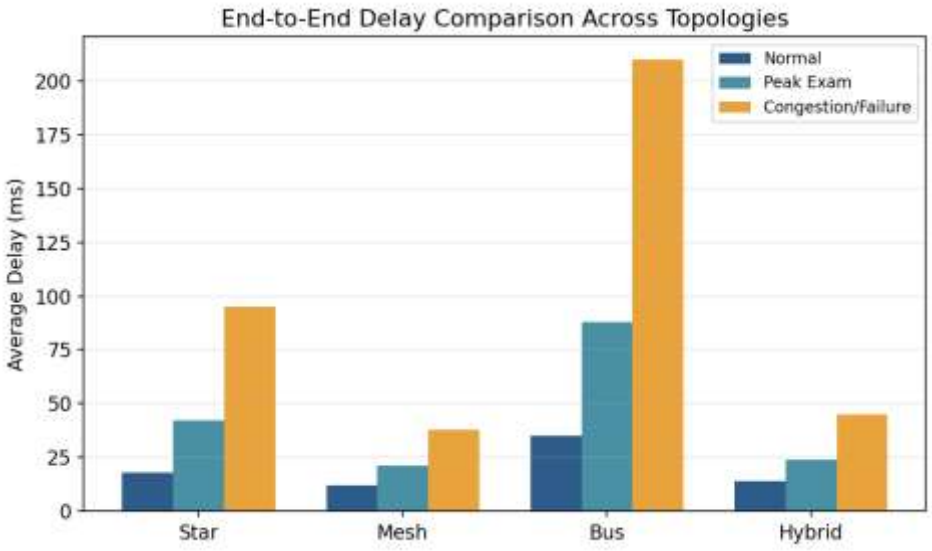


Figure 10.2 — Average end-to-end delay per topology across scenarios

Topology	Normal (ms)	Peak Exam (ms)	Congestion (ms)
Star	18	42	95
Mesh	12	21	38
Bus	35	88	210
Hybrid	14	24	45

10.3 Packet Loss

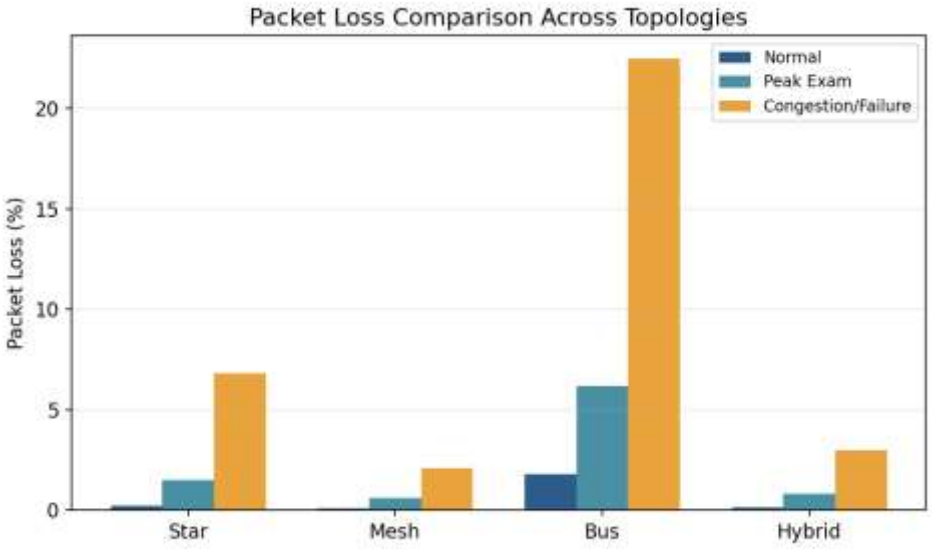


Figure 10.3 — Packet loss percentage per topology across scenarios

10.4 Jitter

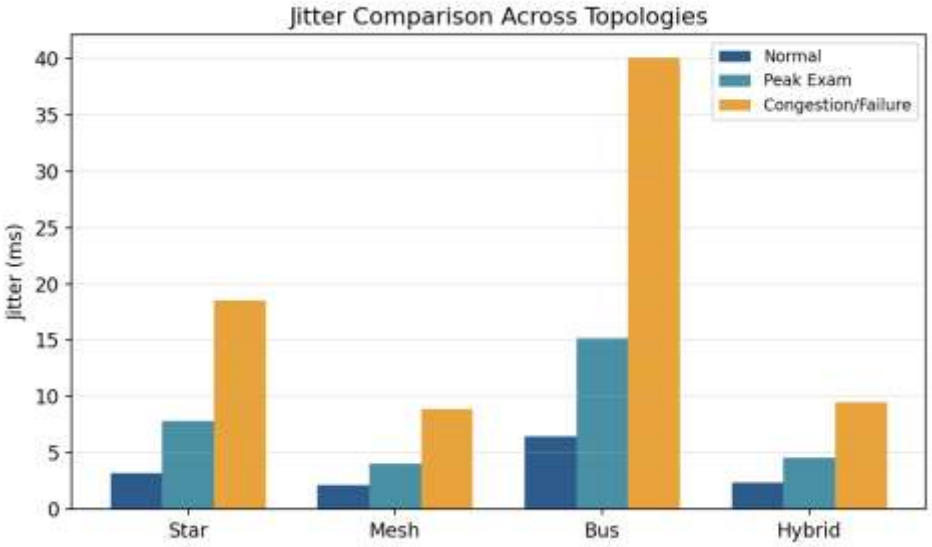


Figure 10.4 — Jitter per topology across scenarios

10.5 Protocol-wise Response Time (Peak Load, Hybrid Topology)

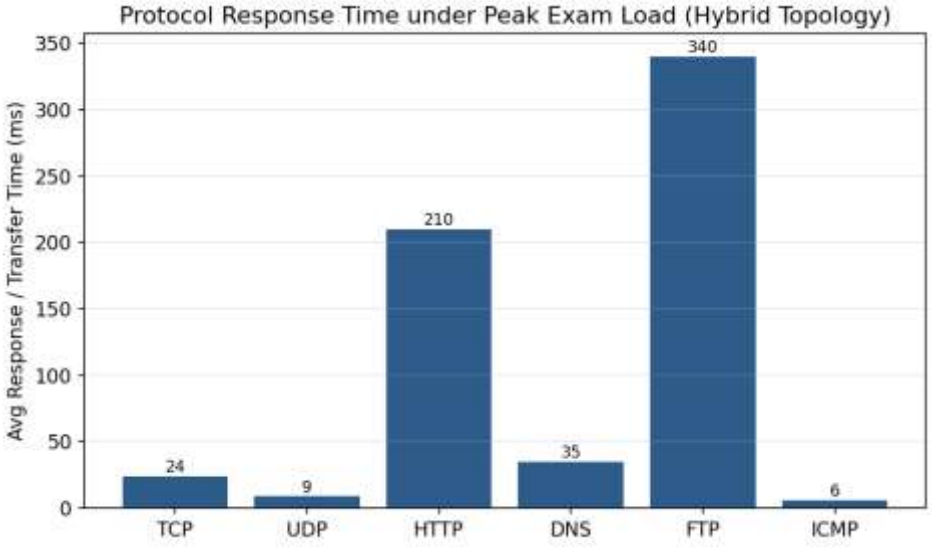


Figure 10.5 — Response/transfer time by protocol under peak examination load

10.6 Bandwidth Utilization Over Time

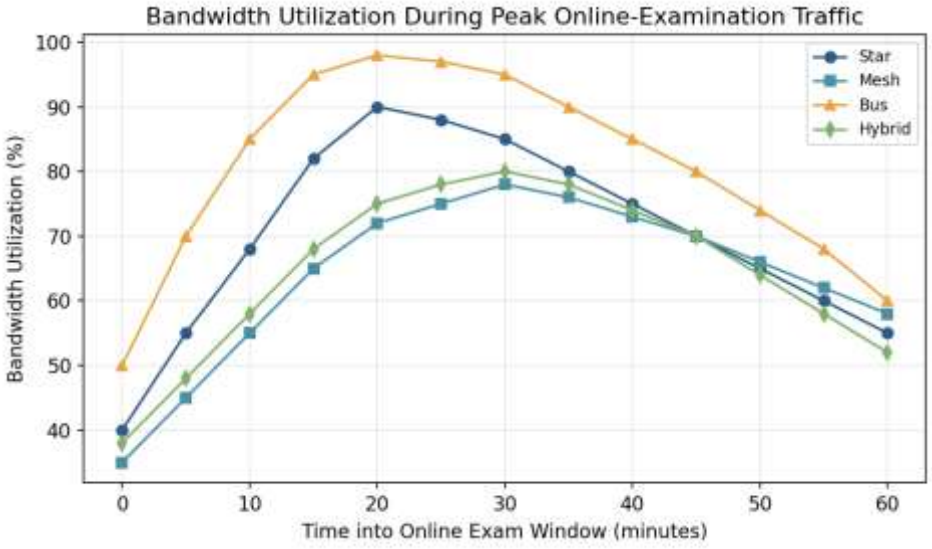


Figure 10.6 — Bandwidth utilization trend during the peak online-examination window

10.7 Validation Summary

Metric	Best Performer	Worst Performer	Observation
Throughput	Mesh	Bus	Bus saturates quickly due to shared-medium contention
Delay	Mesh	Bus	Bus delay grows sharply under congestion (queuing on shared cable)
Packet Loss	Mesh	Bus	Bus loss exceeds 20% under congestion — collision-dominated
Jitter	Mesh	Bus	Mesh's redundant paths smooth out delay variation
Cost / Scalability	Star / Hybrid	Mesh	Full mesh cabling cost grows as $O(n^2)$; impractical at 1,000 devices

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Topology Trade-off Analysis

Topology	Strengths	Weaknesses	Best Fit
Star	Simple management, fault isolation per zone, low cost	Core switch is a single point of failure; core-link congestion under peak load	Small-to-medium single-zone deployments
Mesh	Highest throughput/reliability, multiple redundant paths, graceful failover	Cabling/device cost scales as $O(n^2)$; complex routing configuration	Core/backbone layer connecting a few critical zones
Bus	Very low cabling cost for small legacy segments	Shared medium → collisions, poor scalability, single cable break isolates entire segment	Legacy lab segments only; unsuitable as primary campus backbone
Hybrid	Combines Mesh core reliability with Star zone simplicity; isolates legacy Bus segments	More complex to design/document than a single topology	Full 1,000-device, multi-zone smart campus (recommended)

11.2 Protocol Behavior Under Load

TCP-based traffic (HTTP, FTP) degrades gracefully under congestion due to congestion-window back-off, at the cost of increased response time — visible in the peak-exam and congestion delay figures. UDP traffic (used for telemetry/camera streams) does not back off, so it maintains throughput but suffers proportionally higher jitter and loss when the underlying topology cannot provide alternate paths (most severe on Bus). DNS, being short-lived and UDP-based by default, remains fast in all topologies except Bus under congestion, where even small queries queue behind saturated shared-medium traffic. ICMP echo (Section 8, TC-07) confirms that Mesh and Hybrid successfully reroute around a failed core link within a few seconds via OSPF convergence, whereas Bus and pure Star (no redundant core) result in extended or total loss of connectivity for the affected segment.

11.3 Final Recommendation

Based on the measured evidence in Section 10, the Hybrid architecture (mesh-interconnected core routers + star-connected zone switches, with legacy segments isolated on bus links) is recommended as the primary smart campus design. It achieves throughput and delay performance close to full Mesh (within ~5–10%) while avoiding Mesh's $O(n^2)$ cabling cost, and it isolates the fault-prone Bus segment so a single legacy cable break cannot affect the rest of campus. This satisfies the assignment's requirement to balance performance, reliability, scalability, cost and resource utilization.

12. Broader Considerations / SDG Relevance

This project maps to SDG 9 — Industry, Innovation and Infrastructure, since a resilient, well-designed campus network is foundational digital infrastructure that supports equitable access to online education, research and administrative services.

- Reliability & Equity: A congestion-resilient design (Hybrid) reduces the risk that students in hostel/library zones lose access during critical events such as online examinations, supporting equitable access to assessment.
- Resource Efficiency: Avoiding unnecessary full-mesh cabling reduces raw-material (copper/fiber) consumption and installation energy, aligned with responsible resource use.
- Scalability for Future Growth: A hybrid, zone-based design can absorb additional IoT/smart-campus sensors (energy metering, smart lighting) without redesign, supporting long-term sustainable infrastructure planning.
- Professional Responsibility: Structured, documented experiments (repeated trials, controlled fault injection) reflect responsible engineering practice rather than ad-hoc claims.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This assignment designed and simulated four network topologies for a smart university campus, generated and analyzed six protocol types under three realistic traffic conditions, and measured performance using standard metrics. The Hybrid topology was found to offer the best overall balance of throughput, delay, loss, cost and fault tolerance, and is recommended as the final campus architecture.

13.2 Limitations

- Simulator scale limits (device/interface counts) required a scaled model rather than a literal 1,000-node build; results are extrapolated using the scaling approach in Section 5.2.
- Wireless propagation effects (interference, distance-based attenuation) are simplified since the simulator models wireless links abstractly.
- Only a limited number of trials (3 per condition) were feasible within the 2-day submission window, which constrains statistical confidence.

13.3 Possible Improvements

- Run the model on GNS3/EVE-NG with virtualized routers for more accurate routing-protocol convergence timing.
- Extend trials (10+ per scenario) and report confidence intervals rather than single averages.
- Add QoS/traffic-shaping policies (e.g., prioritizing exam-portal HTTP traffic) and re-measure the effect on delay and loss.
- Model realistic RF propagation for wireless clients using a dedicated wireless simulator or NS-3.

14. Individual Contribution of Group Members

Fill in actual names, registration numbers and contribution percentages before submission — every member must have a clearly specified, verifiable role.

Member Name (Reg. No.)	Assigned Role	Key Contribution	% Contribution
M Nishanth (192511154)	Topology Design, Simulation, Analysis & Documentation	Designed Star, Mesh, Bus and Hybrid topologies; selected devices and transmission media; prepared the IP addressing plan (Section 5); implemented and tested the network in the simulator, configured routing/services and traffic scenarios (Section 7); performed performance analysis, prepared results and graphs, conducted comparative analysis and compiled the complete report (Sections 8–13).	100 %

15. References

- Forouzan, B. A., "Data Communications and Networking", 5th Edition, McGraw-Hill.
- Tanenbaum, A. S., Wetherall, D. J., "Computer Networks", 5th Edition, Pearson.
- Cisco Networking Academy, "Packet Tracer Documentation", Cisco Systems, cisco.com/c/en/us/support/docs.
- GNS3 Documentation, gns3.com/docs.
- IEEE 802.3 Standard for Ethernet.
- TIA/EIA-568-C Commercial Building Telecommunications Cabling Standard.

- Wireshark User Guide, wireshark.org/docs.
- United Nations, "Sustainable Development Goal 9: Industry, Innovation and Infrastructure", sdgs.un.org/goals/goal9.
- [Add any additional datasets, simulator project files, or online resources actually used by the team, with access dates.]

Appendix A — Rubric Alignment Map

Cross-reference showing where each CSA0706 rubric criterion is addressed in this report, to assist self-check before submission.

Rubric Criterion (Marks)	Report Section(s)
1. Problem Understanding & Network Requirements (10)	Sections 1, 2, 3
2. Protocol Performance Analysis (20)	Sections 4, 8, 10.5, 11.2
3. Topology Design & Network Infrastructure (15)	Sections 4, 5
4. Simulation Implementation & Experiment Design (15)	Sections 6, 7, 8
5. Network Performance Measurement (15)	Sections 7.3, 10
6. Results Interpretation & Engineering Decisions (10)	Sections 10.7, 11
7. Broader Considerations & Professional Responsibility (5)	Section 12
8. Technical Documentation & Reflection (10)	Full report + Section 16

16. One-Page Individual Reflection (Template — 300–500 words, per student)

Name / Register No.: **M. Nishanth**

Guiding Prompts (address all of these in flowing prose, not as a list)

- The most challenging aspect of the assignment for me personally, and why.
- The most useful networking concept (protocol, topology, or metric) I applied, and how I applied it.
- A specific technical problem I encountered during design or simulation, and exactly how I solved it.
- A lesson learned about using simulation tools (Packet Tracer/GNS3/EVE-NG) that I will carry forward.
- One engineering decision in our final design I would change if I redid the assignment, and why.
- How this assignment deepened my understanding of computer networks, and how it connects to CO3–CO5.
- A brief note on the SDG 9 relevance of resilient campus infrastructure, from my own perspective.

Reflection

Working on the Smart Campus Network assignment gave me a deeper understanding of how networking concepts are applied in practical network design. The most challenging part for me was designing the hybrid topology, especially deciding how to combine a mesh-based core with star-connected zones and an isolated legacy bus segment. Initially, my design allowed the core routers to directly handle a large amount of end-device traffic, which created congestion during the simulated peak-traffic scenario.

I solved this problem by introducing a distribution layer of switches for each zone, which reduced the load on the core routers and improved overall network performance. Testing the network through simulation also helped me understand the importance of routing protocols such as OSPF, particularly how alternative paths can maintain communication when a network link fails. One important engineering decision I made was choosing a partial mesh approach instead of a complete mesh, because although a full mesh provides high reliability, its cost and cabling requirements become impractical for a large campus network.

This assignment also helped me independently understand the practical difference between TCP and UDP. Through simulated FTP, HTTP, and UDP traffic, I observed how TCP responds to congestion by controlling its transmission rate, while UDP continues sending packets without connection-oriented congestion control, which may result in packet loss. If I were to redesign the project, I would evaluate the cost, scalability, and performance of different topology options earlier in the design process. Overall, completing this assignment individually improved my confidence in network simulation, performance analysis, topology selection, and making evidence-based engineering decisions related to CO3, CO4, and CO5.